

LAB 4 – VIBE CODING

Fruit Catcher — Submission Evidence & Deliverables

Name : Darshil Kachhadiya

SRN : PES1UG25CS820

Updated Repository: https://github.com/Darshil085/lab4_fruit_catcher

Final code location: Lab-4/

1. Deliverable Links

Updated Lab Repository: https://github.com/Darshil085/lab4_fruit_catcher

Before Video:

https://drive.google.com/file/d/1wb3e1R9bSkvWg5S3mN9RAVKHhdMwz0j6/view?usp=drive_link

After Video:

https://drive.google.com/file/d/15_UhZuTN669bcHKXZ2MXPgpHZo99IFSL/view?usp=drive_link

Prompt Evidence / Chat History: [PASTE PUBLIC CHAT HISTORY / PROMPT EVIDENCE LINK HERE]

2. Final Repository Structure

```
lab4_fruit_catcher/  
└── Lab-4/  
    ├── game/  
    │   ├── basket.py  
    │   ├── fruit.py  
    │   └── game_engine.py  
    ├── main.py  
    └── README.md
```

3. Tasks Completed

Task 1 — Fix missed-fruit scoring and life deduction — Commit 3a9a4ab — PASS

Task 2 — Add rotten fruit / hazard objects — Commit d2c64f2 — PASS

Task 3 — Add dynamic falling/spawn difficulty — Commit f7e9871 — PASS

Task 4 — Add fruit splash particle effects — Commit 905f1ce — PASS

Structure — Organize code under Lab-4/ — Commit b03ece5 — PASS

4. Bug Fixed — Missed Fruit

A missed fruit now deducts one life instead of increasing the score.

```
if fruit.is_missed(self.height):
    self.lives -= 1
    self.fruits.remove(fruit)
    if self.lives <= 0:
        self.game_state = "GAME_OVER"
```

5. Feature — Rotten Fruit / Hazard

A hazard is generated separately from normal fruit. Catching it deducts one life.

```
if basket_rect.colliderect(fruit.rect):
    if fruit.is_hazard:
        self.lives -= 1
        if self.lives <= 0:
            self.game_state = "GAME_OVER"
    else:
        self.score += 1
```

6. Feature — Dynamic Difficulty

As score increases, spawn delay decreases and newly spawned objects move faster.

```
difficulty_level = self.score // 5
self.spawn_delay = max(300, 750 - difficulty_level * 100)
fruit = Fruit(self.width)
fruit.speed += difficulty_level * 0.5
```

7. Feature — Splash Particles

Splash droplets appear when an object is caught or reaches the bottom.

```
def create_splash(self, x, y):
    for _ in range(10):
        self.particles.append({
            "x": x, "y": y,
            "vx": random.uniform(-3, 3),
            "vy": random.uniform(-4, -1),
            "life": 30
        })
```

8. Testing Evidence

- ☐ Catch normal fruit → Score +1; lives unchanged — PASS
- ☐ Miss normal fruit → Life -1; score unchanged — PASS
- ☐ Catch hazard → Life -1 — PASS
- ☐ Lose all lives → GAME OVER — PASS
- ☐ Press R → restart — PASS
- ☐ Increase score → difficulty increases — PASS

- ☐ Catch object → splash appears — PASS
- ☐ Object reaches bottom → splash appears — PASS
- ☐ Run from Lab-4/main.py → application starts — PASS

9. Prompt Evidence / Chat History

Prompt 1: Analyze the GitHub repository and tell me exactly what I need to do to finish and submit the assignment.

Prompt 2: Task 1 code changed. What is next?

Prompt 3: Just give me the full code for game_engine.py with proper formatting.

Prompt 4: Move to task 4.

Prompt 5: Are those videos okay?

Public Chat History / Prompt Evidence: <https://chatgpt.com/share/6ac67666-0c34-83ee-8701-d0aab956de70>

10. Video Evidence

Before: https://drive.google.com/file/d/1wb3e1R9bSkvWg5S3mN9RAVKHhdMwz0j6/view?usp=drive_link

After: https://drive.google.com/file/d/15_UhZuTN669bcHKXZ2MXPgpHZo99IFSL/view?usp=drive_link

11. Final Submission Checklist

- ☐ Updated GitHub repository link
- ☐ Code under Lab-4/
- ☐ Separate commits for Tasks 1–4
- ☐ Public Before video
- ☐ Public After video
- ☐ Public prompt/chat evidence
- ☐ PDF/Word evidence attached